# Übungsblatt 5: Serielle Bussysteme

**Mikrocomputertechnik 3 (MCT3)**

**Musterlösung**

## Zielsetzung

Sie vertiefen Vorlesung 5: UART/SPI/I2C (Clock Drift, Pin-Skalierung, Open-Drain), differentielle Übertragung gegen EMI sowie die praktische I2C-Sequenz zum Aufwecken und Auslesen eines Sensors.

**Bearbeitungszeit:** ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 10 P, Aufgabe 2: 12 P (Gesamt: 22 P)

## Teil 1: Theorie — Serielle Busse und differentielle Signale

### Aufgabe 1.1: UART, SPI und I2C im Vergleich

Die On-Board-Kommunikation wird primär durch UART, SPI und I2C dominiert.

1. Erklären Sie das Problem des *Clock Drifts* bei asynchronen Bussen (UART) und warum Start- und Stop-Bit bei jedem Frame essenziell sind.
2. Vergleichen Sie die Topologien von SPI und I2C. Warum skaliert SPI bei vielen Teilnehmern auf der Platine (Pin-Bedarf der CPU) deutlich schlechter als I2C?
3. I2C nutzt elektrisch Open-Drain mit Pull-up-Widerständen statt aktiver Push-Pull-Treiber wie bei SPI. Warum verhindert das Kurzschlüsse, wenn mehrere Geräte gleichzeitig treiben wollen — und warum ist I2C dadurch langsamer als SPI?

**Musterlösung Aufgabe 1.1**

1. UART hat keine gemeinsame Taktleitung: Sender und Empfänger müssen dieselbe Baudrate nutzen. Quarze weichen (Temperatur, Toleranz) ab → die internen Abtastuhren laufen auseinander (*Clock Drift*). Das Start-Bit (fallende Flanke) kalibriert die Empfängeruhr für jedes Byte neu. Das Stop-Bit (HIGH) stellt den Ruhezustand wieder her, damit die nächste fallende Flanke als Start erkennbar bleibt.
2. SPI nutzt typisch Stern-Routing mit eigener Chip-Select-Leitung (`CS`) pro Slave — jeder zusätzliche Slave braucht einen weiteren GPIO an der MCU. Beispiel grob: 10 Slaves → 3 Busleitungen + 10×`CS` = 13 Pins. I2C ist ein adressierter Multi-Drop-Bus: unabhängig von der Gerätezahl bleiben zwei MCU-Pins (`SDA`, `SCL`).

3. I2C-Geräte treiben die Leitung nie aktiv auf HIGH; sie ziehen nur aktiv auf LOW oder lassen los (hochohmig). Pull-ups ziehen dann sanft auf HIGH. Treibt ein Gerät LOW und ein anderes „sendet 1“ (loslassen), fließt Strom nur über den Pull-up nach Masse — kein Kurzschluss zweier starker Push-Pull-Treiber. Nachteil: der passive Pull-up lädt parasitäre Leitungskapazität langsamer um als SPI-Push-Pull → flachere Flanken, niedrigere Maximalfrequenz.

### Aufgabe 1.2: Industrieumgebungen und Electromagnetic Interference (EMI)

Single-Ended-Busse (z. B. UART/I2C auf kurzen Board-Leitungen) scheitern oft bei langen Kabeln in rauen Umgebungen (z. B. Fabrikhallen).

1. Warum führt eine Gleichtaktstörung (Spike) durch das Magnetfeld eines Elektromotors bei einem UART-Kabel (Single-Ended) oft zu Bitfehlern, bei RS-485 oder CAN (differentiell) jedoch typisch nicht?
2. Welche physikalische Kabeleigenschaft (Stichwort: *Twisted Pair*) ist für die Störungsunterdrückung bei differentieller Übertragung mitentscheidend?

**Musterlösung Aufgabe 1.2**

1. Single-Ended wertet den Pegel gegen GND aus. Eine Störung auf der Datenleitung kann LOW als HIGH erscheinen lassen. Ein differentieller Empfänger bildet $V_A - V_B$. Dieselbe Störung auf beiden Adern lässt die Differenz unverändert, verschiebt aber den Common-Mode-Pegel. Der erlaubte Common-Mode-Bereich steht im Transceiver-Datenblatt; außerhalb bleiben Fehler und Schäden möglich.

2. *Twisted Pair* nähert eine ähnliche Einkopplung an. Unsymmetrie, Leitungsführung und der reale Empfänger heben die Unterdrückung nicht vollständig auf.

## Teil 2: Praxis — Sensor-Kommunikation via I2C

**Szenario:** Sie initialisieren und lesen in C einen I2C-Beschleunigungssensor (IMU, MPU6050-ähnlich). Hilfsfunktionen abstrahieren den Controller.

**Gegeben:**

- 7-Bit-Adresse des Sensors: `0x68`
- Power-Management-Register: Subadresse `0x6B` (Schreiben von `0` weckt den Chip)
- X-Achse roh: `0x3B` (High-Byte), `0x3C` (Low-Byte)
- Hilfsfunktionen: `i2c_send_start()`, `i2c_send_stop()`, `wait_for_ack_timeout()`, `i2c_send_byte(uint8_t data)`, `i2c_read_byte_ack()`, `i2c_read_byte_nack()`
- `wait_for_ack_timeout()` liefert falsch bei Address-NACK, unerwartetem Data-NACK oder abgelaufener Deadline. Ein fehlender Sensor darf keine Endlosschleife erzeugen.
- Der NACK nach dem letzten gelesenen Byte ist der normale Abschluss und kein Fehler.

Adressbyte: 7-Bit-Adresse um eins nach links schieben, R/W im LSB (`0` = Write, `1` = Read).

### Aufgabe 2.1: Sensor aufwecken (I2C Write)

Vervollständigen Sie die Funktion. Achten Sie auf die korrekte Bildung des Adressbytes.

```
void sensor_wakeup(void) {
    i2c_send_start();

    /* 1. Geräteadresse im Write-Modus (R/W = 0) */
    /* TODO: i2c_send_byte(...); */
    wait_for_ack_timeout();

    /* 2. Register-Adresse Power-Management */
    /* TODO: i2c_send_byte(...); */
    wait_for_ack_timeout();

    /* 3. Datenwert zum Aufwecken */
    /* TODO: i2c_send_byte(...); */
    wait_for_ack_timeout();

    i2c_send_stop();
}
```

### Musterlösung Aufgabe 2.1

```
void sensor_wakeup(void) {
    i2c_send_start();

    /* 7-Bit-Adresse << 1, Write-Bit = 0 */
    i2c_send_byte((0x68u << 1) | 0u);
    wait_for_ack_timeout();

    /* PWR_MGMT_1 */
    i2c_send_byte(0x6Bu);
    wait_for_ack_timeout();
```

```
    /* 0 = Sleep aus / aufwecken (vereinfachtes Modell) */
    i2c_send_byte(0x00u);
    wait_for_ack_timeout();

    i2c_send_stop();
}
```

### Aufgabe 2.2: Nur das High-Byte der X-Achse

`read_sensor_x_high()` liest genau ein Byte ab `0x3B`. Das ist nicht der 16-Bit-Wert. Zuerst den Registerzeiger per Write setzen, dann mit *Repeated Start* auf Read umschalten. Der NACK nach diesem einen Byte ist hier der vorgesehene Abschluss.

```
uint8_t read_sensor_x_high(void) {
    i2c_send_start();

    /* 1. Adresse Write: Registerzeiger setzen */
    /* TODO: i2c_send_byte(...); */
    wait_for_ack_timeout();

    /* 2. Register-Adresse X-High */
    /* TODO: i2c_send_byte(...); */
    wait_for_ack_timeout();

    /* 3. Repeated Start, Adresse Read (R/W = 1) */
    i2c_send_start();
    /* TODO: i2c_send_byte(...); */
    wait_for_ack_timeout();

    uint8_t data = i2c_read_byte_nack();
    i2c_send_stop();
    return data;
}
```

### Musterlösung Aufgabe 2.2

```
uint8_t read_sensor_x_high(void) {
    i2c_send_start();

    i2c_send_byte((0x68u << 1) | 0u);
    wait_for_ack_timeout();

    i2c_send_byte(0x3Bu); /* ACCEL_XOUT_H */
    wait_for_ack_timeout();

    i2c_send_start(); /* Repeated Start */
    i2c_send_byte((0x68u << 1) | 1u);
    wait_for_ack_timeout();

    uint8_t data = i2c_read_byte_nack();
    i2c_send_stop();
    return data;
}
```

### Aufgabe 2.3: Endianness und Bit-Shifting

Der Sensor liefert 16-Bit-Beschleunigungswerte als High- und Low-Byte (*Big-Endian*). Bauen Sie aus `hi` und `lo` einen vorzeichenbehafteten 16-Bit-Wert.

Hinweis: Zusätzlich angenommen ist `read_sensor_x_low()`, die analog Register `0x3C` liest.

```
uint8_t hi = read_sensor_x_high(); /* z. B. 0x01 */
uint8_t lo = read_sensor_x_low();  /* z. B. 0x2A */

/* TODO: 16-Bit-Wert zusammensetzen */
int16_t x_achse = /* ... */;
```

### Musterlösung Aufgabe 2.3

```
uint8_t hi = read_sensor_x_high(); /* z. B. 0x01 */
uint8_t lo = read_sensor_x_low();  /* z. B. 0x2A */

/* High-Byte << 8, mit Low-Byte verodern; Cast erhält Vorzeichenbit */
int16_t x_achse = (int16_t)(((uint16_t)hi << 8) | lo);
```

(Hinweis: `((uint16_t)hi << 8) | lo` vermeidet Mehrdeutigkeiten bei der Integer-Promotion; das Vorzeichenbit von Bit 15 bleibt nach dem Cast auf `int16_t` erhalten. Zwei getrennte Leseaufrufe können Werte verschiedener Abtastungen mischen. Ein zusammenhängender Burst über `0x3B` bis `0x40` hält die drei Achsen beieinander.)

### Aufgabe 2.4: Sechs Byte als Beschleunigung

Ein Burst liefert `10 00 E0 00 40 00`. Der Bereich $\pm 2\,\mathrm{g}$ ist mit 16384 LSB/g konfiguriert.

1. Bilden Sie die drei vorzeichenbehafteten Rohwerte.
2. Rechnen Sie sie in $g$ um.
3. Welche Ergebnisse liefern die Grenzen `0x0000`, `0x7FFF`, `0x8000` und `0xFFFF`?

### Musterlösung Aufgabe 2.4

1. $X = 0\text{x}1000 = 4096$, $Y = 0\text{xE}000 = -8192$, $Z = 0\text{x}4000 = 16384$.
2. $+0{,}25\,\mathrm{g}$, $-0{,}5\,\mathrm{g}$, $+1\,\mathrm{g}$.
3. 0, 32767, $-32768$, $-1$.

Die Bytes werden unsigned zusammengesetzt und danach als `int16_t` gelesen. Ein Cast eines Bytepuffers auf `int16_t *` legt Bytefolge und Alignment nicht fest. `0x00` und `0xFF` bleiben gültige Messbytes und sind kein Fehlercode.